❶ Derives from ValidationAttribute to ensure that your attribute is used during validation

❷ The attribute takes in an array of allowed currency codes.

❸ The IsValid method is passed the value to validate and a context object.

❹ Tries to cast the value to a string and store it in the code variable

❺ If the value provided isn't a string, is null, or isn't an allowed code, returns an error . . .

❻ . . .otherwise, returns a success result.

As you know from chapter 16, Validation occurs in the filter pipeline after model binding, before the action or Razor Page handler executes. The validation framework calls `IsValid()` for each instance of `ValidationAttribute` on the model property being validated. The framework passes in `value` (the value of the property being validated) and the `ValidationContext` to each attribute in turn. The context object contains details that you can use to validate the property.

Of particular note is the `ObjectInstance` property. You can use this to access the top-level model being validated when you validate a subproperty. For example, if the `CurrencyFrom` property of the `CurrencyConvertModel` is being validated, you can access the top-level object from the `ValidationAttribute` as follows:

```
var model = validationContext.ObjectInstance as CurrencyConverterModel;
```

This can be useful if the validity of a property depends on the value of another property of the model. For example, you might want a validation rule that says that GBP is a valid value for `CurrencyTo` except when `CurrencyFrom` is also GBP. `ObjectInstance` makes these sorts of comparison validations easy.

**NOTE** Although using `ObjectInstance` makes it easy to make model-level comparisons like these, it reduces the portability of your